

COMPUTADORAS
PARADIGMAS
LENGUAJES DE PROGRAMACIÓN

COMPUTADORAS

Todas las computadoras, desde las grandes supercomputadoras hasta las computadoras personales, deben realizar un **conjunto mínimo de funciones** y proporcionar la capacidad para:

1. **Aceptar** entradas
2. **Mostrar** salidas
3. **Almacenar** información en un formato lógico consistente (tradicionalmente binario)
4. **Ejecutar** operaciones aritméticas y lógicas en los datos de entrada o en los almacenados
5. **Supervisar, controlar y dirigir** la operación y secuencia general del sistema.

Las computadoras - Hardware y Software

- **Computadora:**

Dispositivo electrónico utilizado para procesar información y obtener resultados, capaz de ejecutar **cálculos** y **tomar decisiones** a velocidades millones o cientos de millones más rápido que lo que pueden hacerlo los seres humanos.

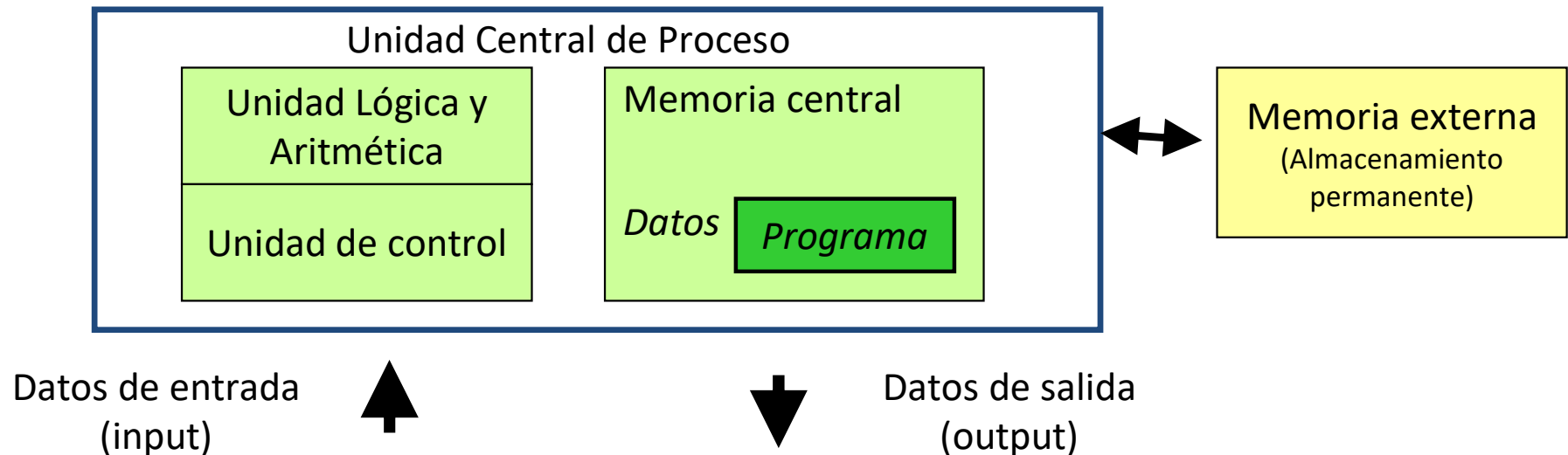
Personales, servidores, minicomputadoras, mainframes y supercomputadoras.

Las computadoras - Hardware y Software

El término **computadora** se emplea para referir a dos partes:

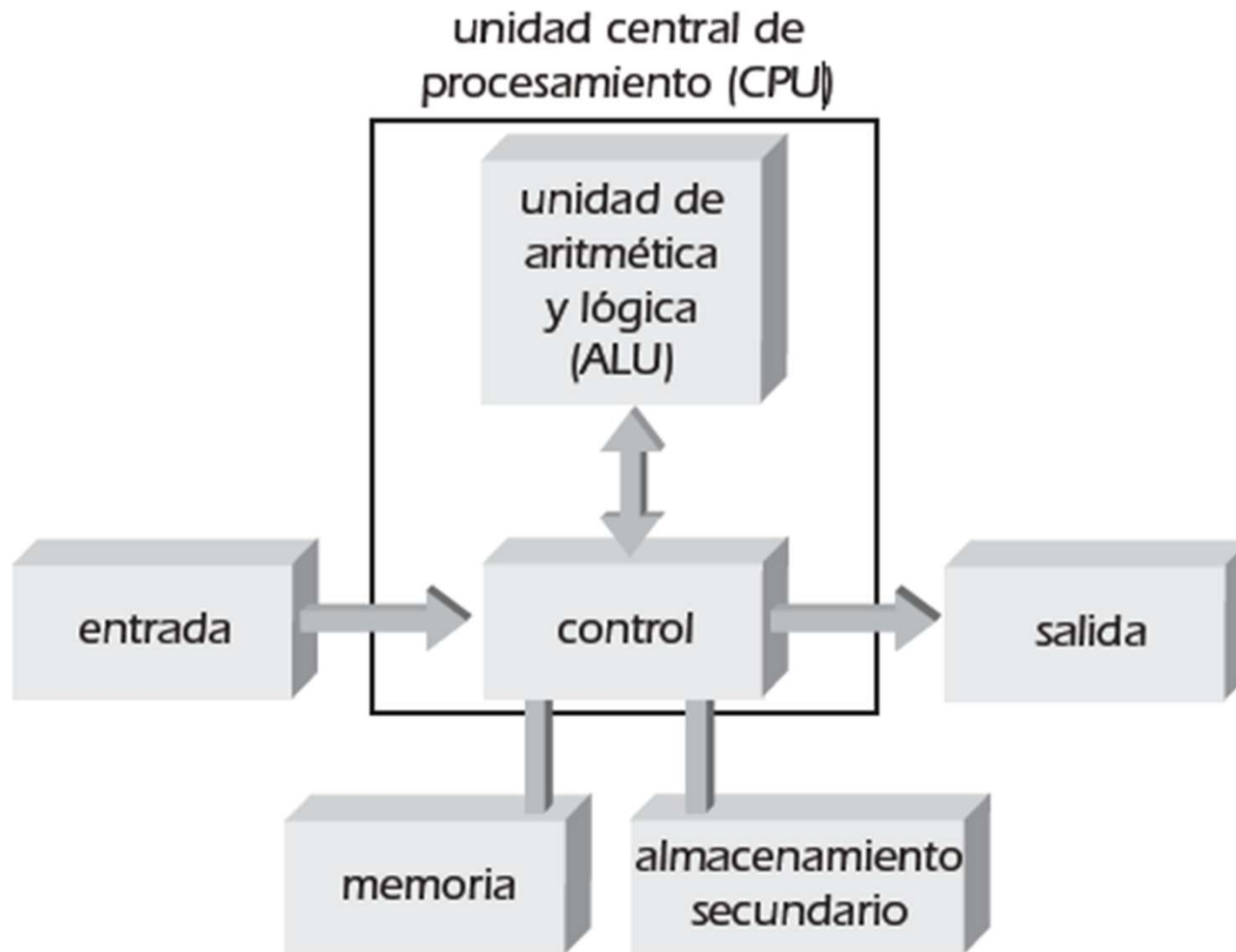
Hardware (la computadora en sí, junto a los dispositivos de E/S) y

Software (conjunto de programas que controlan y dirigen a la computadora para que realice tareas especificadas)



Computadoras

Unidades básicas del hardware de una computadora



Computadoras

La **unidad de aritmética y lógica** (ALU) ejecuta todas las funciones aritméticas y lógicas como adición y sustracción, y las proporcionadas por la computadora.

La **unidad de control** dirige y supervisa la operación general de la computadora. Rastrea el lugar de la memoria donde reside la siguiente instrucción, emite las señales necesarias para leer datos y escribir datos en otras unidades en el sistema y controla la ejecución de todas las instrucciones.

La **unidad de memoria** almacena información en un formato lógico consistente. Tanto instrucciones como datos se almacenan en la memoria, por lo general en áreas separadas y distintas.

La **unidad de entrada y salida** (I/O o E/S) proporciona la interfaz a la que se conectan dispositivos periféricos como teclados, monitores, impresoras , parlantes...

Computadoras

La unidad de aritmética y lógica con la unidad de control se combinan en una sola unidad llamada **unidad central de procesamiento (CPU)**. La mayoría de las señales de control generadas por un programa están dirigidas a la ALU en respuesta a instrucciones aritméticas y lógicas dentro del programa.

Almacenamiento secundario: Debido a que la memoria principal es relativamente cara y volátil (la información se pierde cuando se suspende la energía), no es práctica como un área de almacenamiento permanente para programas y datos. Para esto se usan dispositivos de almacenamiento secundario o auxiliar.

Almacenamiento

- Las computadoras solo entienden un lenguaje compuesto de ceros y unos (**Sistema Binario Digital**). Este lenguaje de máquina utiliza secuencias o patrones de 0 y 1 para componer datos e instrucciones.
- La unidad de información más pequeña que una computadora puede tratar se denomina **bit** (**b**inary **d**igit).
- La memoria central es una zona de almacenamiento organizada en unidades individuales o **celdas**, cuya unidad mínima de información es el *bit* y al conjunto máximo de bits que la computadora es capaz de manejar en un momento dado se le denomina **palabra**.
- Cada celda tiene una **dirección** única que indica su posición relativa en la memoria y los datos que se almacenan en ella constituyen el **contenido** de la celda.
- Las palabras de 8 bits se conocen como **Byte (B)** y a las otras se las reconoce como palabras de 2 bytes (16 bits), de 4 bytes (32 bits), etc.
- Otras **medidas de almacenamiento**: **KB** (aprox 1000B), **MB** (aprox 1000KB),
GB (aprox 1000 MB), **TB** (aprox 1000 GB).

Almacenamiento en computadoras

- Uno o más bytes pueden agruparse en unidades más grandes, llamadas **palabras**, lo cual facilita un acceso más rápido y más extenso a los datos.
- El número de bytes en una palabra determina los valores enteros máximo y mínimo que pueden ser representados por la palabra.

Tabla 1.2 Valores enteros y tamaño de una palabra

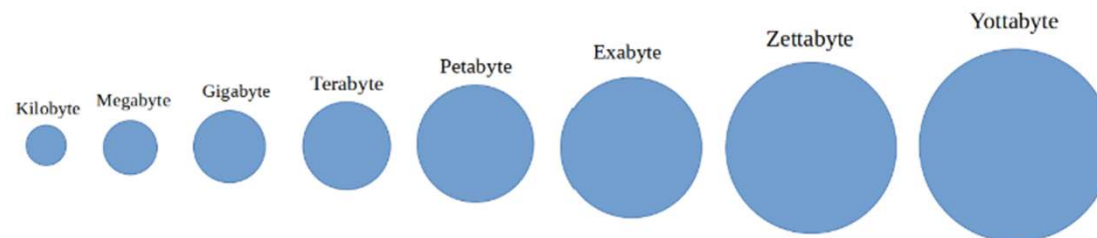
Tamaño de la palabra	Valor del número entero máximo	Valor del número entero mínimo
1 Byte	127	-128
2 Bytes	32 767	-32 768
4 Bytes	2 147 483 647	-2 147 483 648

- Además las computadoras también deben almacenar y transmitir números que contienen puntos decimales, los cuales se conocen en matemáticas como **números reales**.

Medidas de Almacenamiento

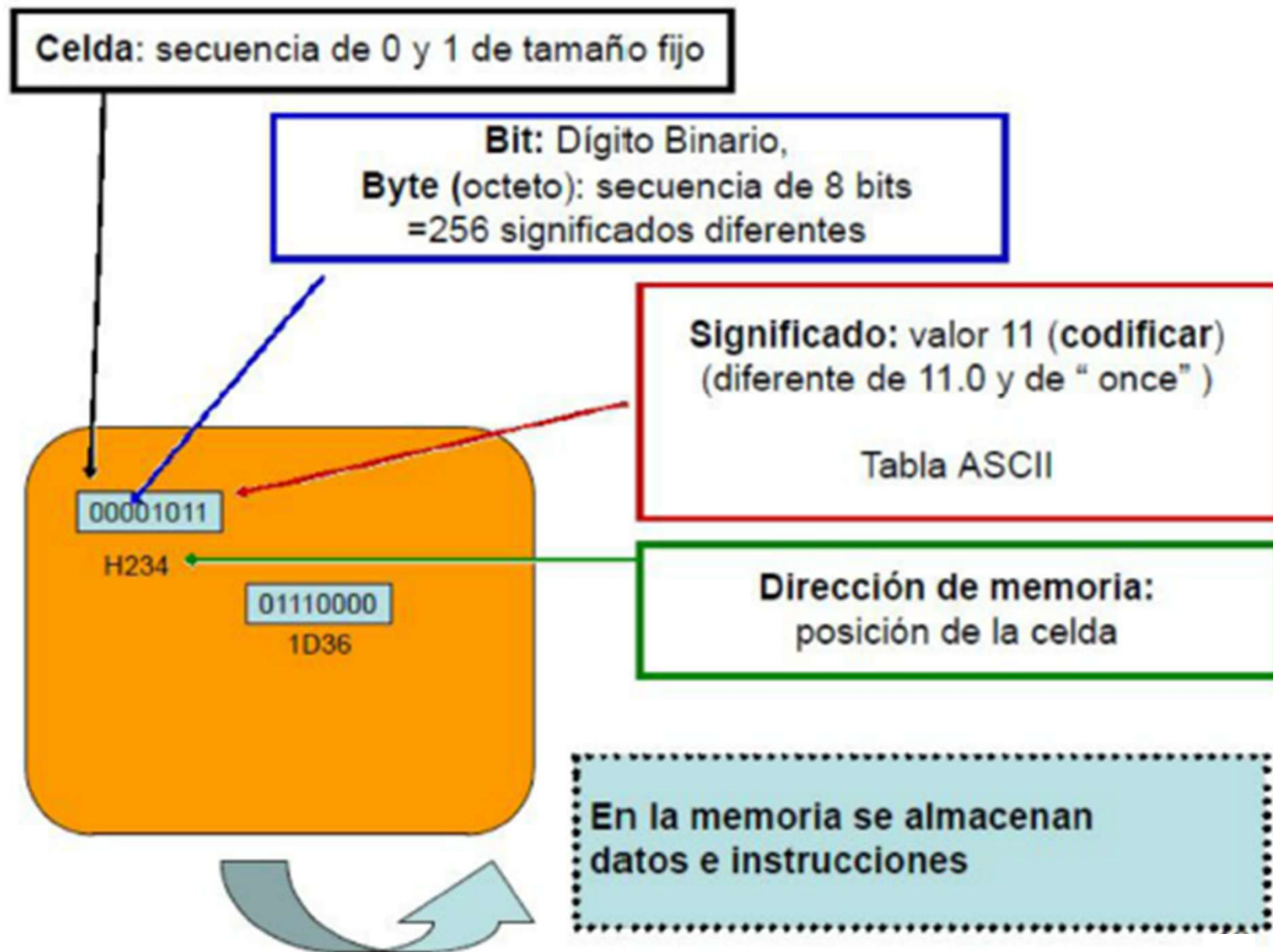
Medida	Símbolo	Equivalente
Byte	<i>byte</i>	8 bits
Kilobyte	<i>KB</i>	1024 bytes
Megabyte	<i>MB</i>	1024 KB
Gigabyte	<i>GB</i>	1024 MB
Terabyte	<i>TB</i>	1024 GB
Petabyte	<i>PB</i>	1024 TB
Exabyte	<i>EB</i>	1024 PB
Zettabyte	<i>ZB</i>	1024 EB
Yottabyte	<i>YB</i>	1024 ZB

Representación gráfica de las unidades de almacenamiento de la información



Un archivo de texto o libro electrónico puede ocupar varios Kilobytes, una foto sacada con nuestro teléfono celular puede ocupar varios Megabytes y el archivo de vídeo de una película puede llegar a varios Gigabytes de información. La cantidad de datos que genera Facebook durante un solo día o la cantidad de información que es capaz de almacenar en un solo año, se expresa en Petabytes. La cantidad de datos que manejan los servidores del buscador Google, ya se mide desde hace años, en Exabytes, y continúa creciendo exponencialmente.

Codificación de información



Memoria y Direcccionamiento

- Los bytes sirven para representar caracteres (letras, números, símbolos especiales) en un código estándar internacional: **ASCII (American Standard Code for Information Interchange)**, base y extendido
- La computadora dispone de **dos tipos de memoria**:
 - ▶ **Central o Principal**: En ella se almacenan: **Datos de entrada** para procesarse, **Programas** que realizarán los procesos, **Resultados** de salida.
 - RAM** (Random Access Memory) almacenamiento temporal de datos y programas. Es volátil. Es muy rápida pero limitada en tamaño.
 - ROM** (Read Only Memory) Viene programada por el fabricante, es permanente y de solo lectura. Tiene los programas básicos de arranque.
 - ▶ **Externa o Secundaria o Auxiliar**: reside en dispositivos y es permanente.

Computadoras

- ▶ Un **byte** consistente en ocho bits, donde cada bit es 0 o 1 y puede representar cualquiera de **256 patrones distintos**
- ▶ Éstos consisten del patrón 00000000 (los ocho interruptores abiertos) al patrón 11111111 (los ocho interruptores cerrados) y todas las combinaciones intermedias posibles de 0 y 1
- ▶ Cada uno de estos patrones puede usarse para representar ya sea una letra del alfabeto, otros caracteres individuales como un signo de peso, una coma, etc., un solo dígito, o números que contienen más de un dígito.
- ▶ La colección de patrones se llama **código de caracteres**. Uno de estos códigos es llamado código ASCII.

Tabla ASCII

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	:	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com

Tabla ASCII

Extended ASCII Codes

128	Ç	144	É	160	á	176	░	192	Ł	208	⌌	224	α	240	≡
129	ü	145	æ	161	í	177	▒	193	ł	209	⌍	225	β	241	±
130	é	146	Æ	162	ó	178	▓	194	Ṁ	210	⌎	226	Γ	242	≥
131	â	147	ô	163	ú	179		195	ṁ	211	⌏	227	π	243	≤
132	ä	148	ö	164	ñ	180	└	196	—	212	⌐	228	Σ	244	∫
133	à	149	ò	165	Ñ	181	┘	197	+	213	Ṛ	229	σ	245	┐
134	â	150	û	166	ª	182	┌	198	Ṃ	214	Ṕ	230	μ	246	÷
135	ç	151	ù	167	º	183	Ṁ	199	Ṓ	215	Ṗ	231	τ	247	≈
136	ê	152	ÿ	168	¿	184	Ṃ	200	Ṕ	216	Ṛ	232	Φ	248	°
137	ë	153	Ö	169	┐	185	Ṓ	201	Ṛ	217	Ṕ	233	⊖	249	·
138	è	154	Û	170	└	186	Ṓ	202	Ṕ	218	Ṛ	234	Ω	250	·
139	ï	155	¢	171	½	187	Ṓ	203	Ṓ	219	■	235	δ	251	√
140	î	156	£	172	¼	188	Ṓ	204	Ṓ	220	■	236	∞	252	∞
141	ì	157	¥	173	¡	189	Ṓ	205	=	221	■	237	φ	253	²
142	Ä	158	£	174	«	190	Ṓ	206	Ṓ	222	■	238	ε	254	■
143	Å	159	ƒ	175	»	191	Ṓ	207	Ṓ	223	■	239	∩	255	

Source: www.LookupTables.com

SOFTWARE

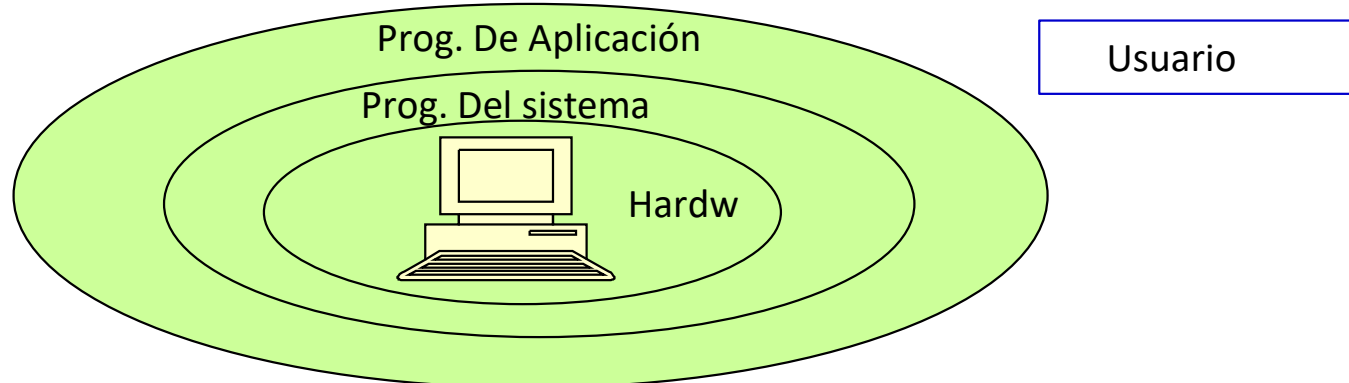
- Un programa de **software** es un conjunto de instrucciones o sentencias para la computadora, que indican las operaciones que debe realizar el hardware.

- **Tipos de software:**

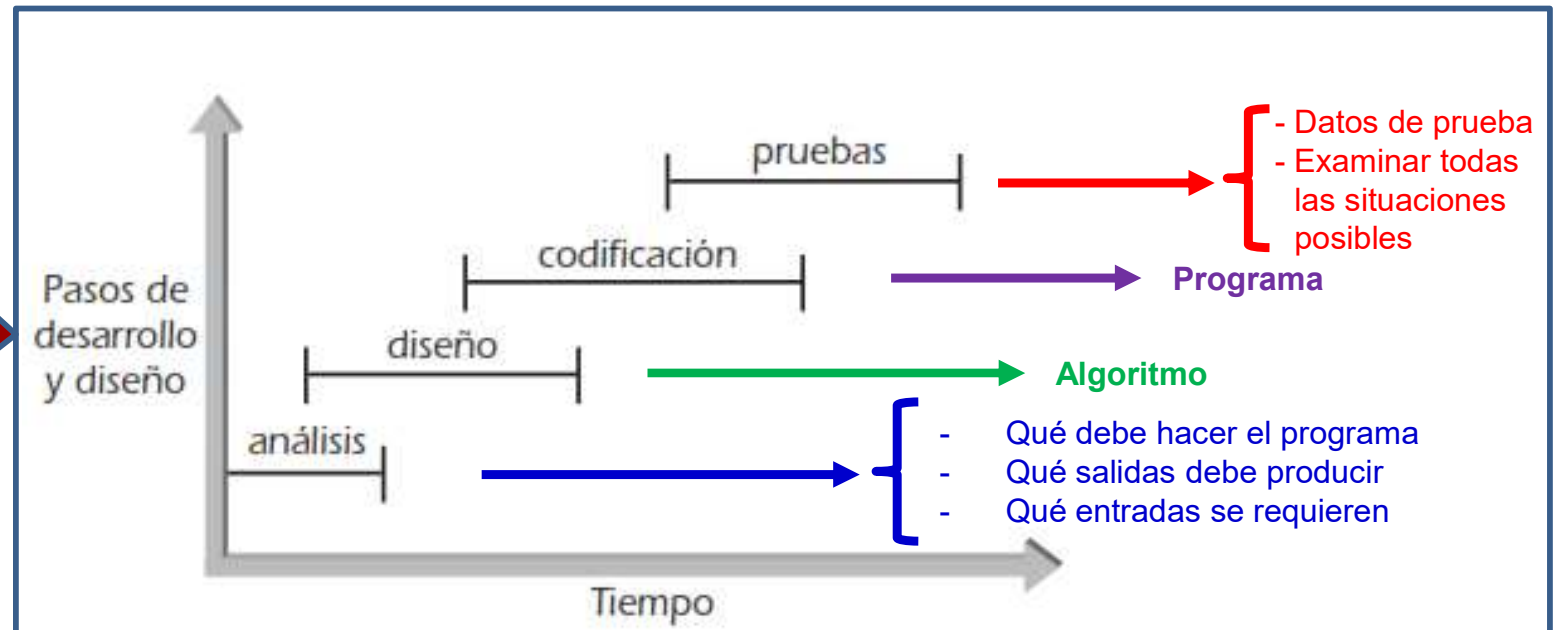
- Del sistema:**

Gestiona los recursos de la computadora (procesador central, enlaces de comunicaciones, periféricos). Coordina las diferentes partes de la computadora y conecta e interactúa entre el software de aplicación y el hardware: **Sistema operativo** + **Utilitarios** (realizan tareas de proceso comunes) + **Compiladores** (convierten los lenguajes de programación a lenguajes de máquina).

De aplicaciones: programas escritos por empresas o usuarios, individuales o en equipo, para realizar una tarea específica por medio de la computadora.



Ciclo de vida del software



LENGUAJES DE PROGRAMACIÓN

- ▶ Los lenguajes de programación se utilizan para escribir programas:

LENGUAJES DE BAJO NIVEL:

- Lenguaje de Máquina: Las instrucciones se codifican como secuencias de dígitos binarios (1ra. generación).
- Lenguajes Ensambladores: utilizan códigos mnemotécnicos (2º generación)

LENGUAJES DE ALTO NIVEL: Amigables al programador (3º generación) - Independientes de la máquina: **FORTRAN, COBOL, Pascal, Basic, C, C++, Java, C#, HTML, PHP, Python**

- Juego de instrucciones (E/S, de cálculo, de control)
- Reglas de sintaxis y semántica

- ▶ Los programas escritos ya sea en lenguajes de alto nivel o en lenguajes ensambladores, necesitan traducirse a código de máquina, a través de otros programas “traductores” (compiladores)
- ▶ El primer paso de la programación suele comenzar con **lenguajes algorítmicos** (seudocódigo)

LENGUAJES DE PROGRAMACIÓN

- La evolución de los lenguajes de programación ha ido en paralelo a los **paradigmas de programación**:
 - Un paradigma de programación es un estilo de desarrollo de programas. Es decir, un modelo para resolver problemas computacionales
 - Es una propuesta tecnológica adoptada por una comunidad de programadores y desarrolladores
 - Representa un enfoque particular o filosofía para diseñar soluciones
 - Difieren unos de otros, en los conceptos y la forma de abstraer los elementos involucrados en un problema, así como en los pasos que integran su solución del problema, en otras palabras, el cómputo
- Los lenguajes de programación clásicos son:
 - ✓ Imperativo (o procedimental),
 - ✓ Funcional,
 - ✓ Lógico,
 - ✓ Orientado a objetos.
 - ✓ Otros (dirigido por eventos, orientado a aspectos,...)

Lenguajes de alto nivel

Nombre	Propósito
FORTRAN	Lenguaje técnico y científico por excelencia. Evolución para programación estructurada.
ALGOL	Admite recursividad.
COBOL	Baja potencia de cálculo. Utilizado en gestión.
RPG/PL1	Gestión.
PASCAL	Programación estructurada.
Ada	Ámbito científico y técnico.
Modula-2	Programación estructurada.
C	Estructurado. Gran potencia de tratamiento de datos.
C++	Estructurado. Gran potencia de tratamiento de datos. Orientado a objetos.
LISP	Lenguaje de tipo funcional.
PROLOG	Usado en Inteligencia Artificial.
Java	De propósito general.

Metodología de Programación Estructurada

Programación estructurada:

Metodología que consiste en escribir programas con las siguientes reglas:

- El programa tiene un **diseño modular**
- Los módulos son diseñados de **modo descendente**
- Cada módulo se codifica utilizando las **tres estructuras de control** básicas:
 - **Secuencia**
 - **Selección**
 - **Repetición**

Metodología de Programación Estructurada

Teorema fundamental de la PE (Böhm y Jacopini, 1966):

“Todo programa propio puede ser escrito usando solamente las estructuras de control básicas: secuencia, selección y repetición”

Programa propio es aquel que:

- Tiene un único punto de entrada y un único punto de salida
- Existen caminos desde la E a la S que se pueden seguir y que pasan por todas las partes del programa
- Todas las instrucciones son ejecutables y no hay bucles infinitos.

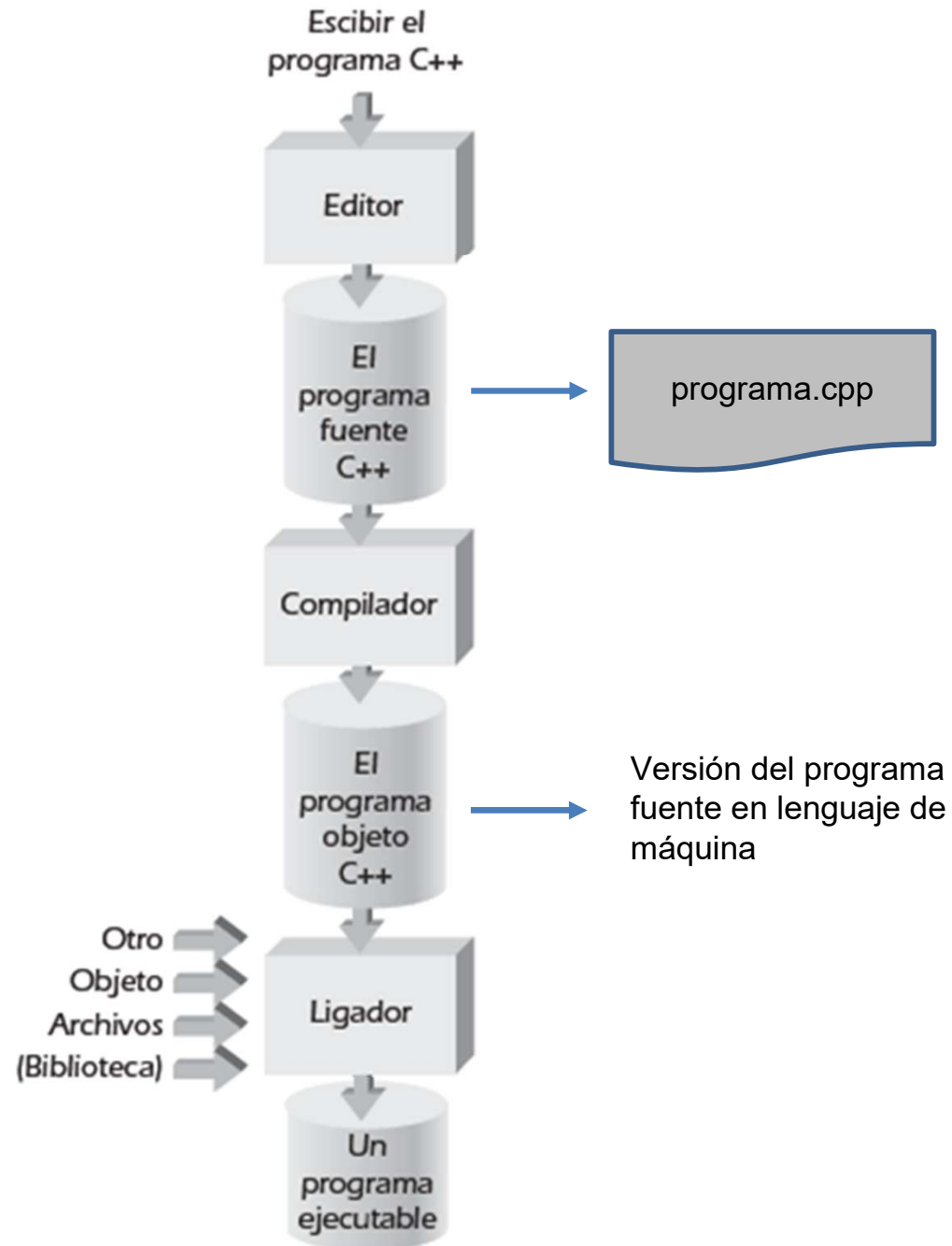
Lenguaje C++

- ▶ El **lenguaje C** es un lenguaje **de procedimiento estructurado** desarrollado en la década de los años 70.
- ▶ Se convirtió en el lenguaje dominante para aplicaciones de ingeniería de la década de los años 80.
- ▶ Tiene un amplio conjunto de capacidades que permite que se escriba como un lenguaje de nivel alto mientras conserva la capacidad de acceso directo a las características del nivel de máquina de una computadora.
- ▶ **El lenguaje C++** fue desarrollado a principios de la década de los años 80 para crear un lenguaje de programación **orientado a objetos**.
- ▶ C++ conservó el conjunto extenso de capacidades estructuradas de C, pero agregó su propia orientación a objetos para convertirse en un verdadero lenguaje de programación de uso general.

Compilación y Ejecución de un Programa

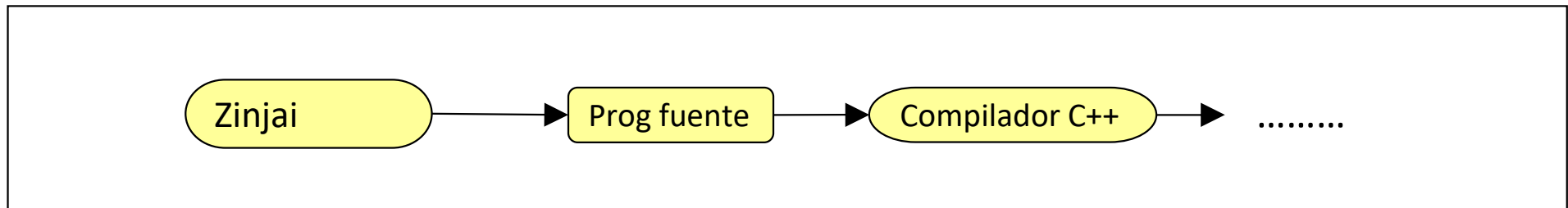
- El algoritmo debe escribirse mediante un programa editor y almacenarse en disco (archivo de programa), convirtiéndose en programa fuente.
- El programa fuente se traduce a lenguaje máquina. Esto lo realiza el compilador con el sistema operativo.
- Si en la compilación se encuentran errores (errores de compilación), se vuelve a editar el programa, se corrigen los errores y se compila de nuevo. Si no hay errores se obtiene el programa objeto (versión en lenguaje de máquina del código fuente).
- Luego hay que realizar un montaje o enlace (link), que es una combinación del programa objeto con las bibliotecas del compilador. Esto produce un programa ejecutable (programa completo en lenguaje de máquina).
- El ejecutable se puede ejecutar o correr (run). Si no hay errores de ejecución, se obtendrá la salida del programa.

Creación de un programa C++ ejecutable



C++: Lenguaje compilado

- Antes de poder **ejecutar** el programa que escribimos, tenemos que **compilar** el código fuente (archivo **.cpp**) con un compilador de C++
- El compilador lee y analiza todo el programa. Si no hay errores lo traduce a código de máquina; sino muestra los errores detectados
- Los compiladores se pueden llamar habitualmente desde la línea de comandos, aunque nosotros usaremos un **entorno de programación (IDE)** que nos facilita la tarea



Lenguaje C++

En general, un programa C++ suele estar escrito en **varios archivos**.

Durante el proceso de compilación estos archivos serán combinados adecuadamente y traducidos a código objeto, obteniendo el programa ejecutable.

Para programas sencillos, bastará un único archivo cuya **extensión** será **“.cpp”**

El archivo suele comenzar con unas líneas para incluir las definiciones de los **módulos de biblioteca** que utilice nuestro programa, e irá seguido de **declaraciones y definiciones de tipos, de constantes y de subprogramas**.

El programa debe contener **un subprograma especial (la función main)** que indica dónde comienza la ejecución. Las instrucciones contenidas en dicha función main se ejecutarán una tras otra hasta llegar a su fin.

La función main devuelve un valor que indica si el programa ha sido ejecutado correctamente o, por el contrario, ha ocurrido un error.

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Hola mundo" << endl;
    return 0;
}
```

Estructura de un Programa en C++

Directivas del preprocesador:

```
# include <xxxx.h>  
# define .....
```

Declaraciones globales:

- Prototipos de funciones
- Variables comunes a todas las funciones del programa

Función principal:

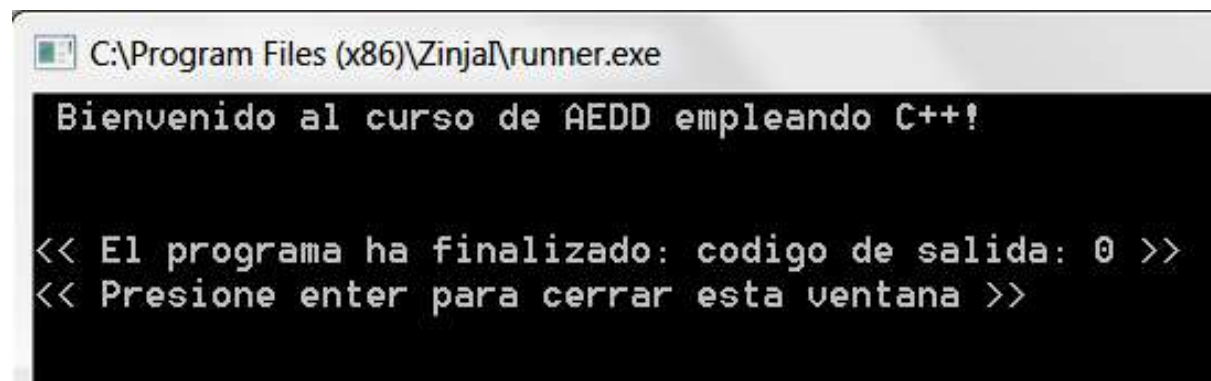
```
int main ( )  
{  
    - Declaraciones locales  
    - Sentencias  
}
```

Definiciones de otras funciones:

```
tipo-retorno funcion1 (..... )  
{  
    .....  
}
```

Estructura de un Programa en C++

```
/* Programa de prueba para mostrar sintaxis de C++ */  
  
#include <iostream>  
using namespace std;  
  
/* Este programa muestra en pantalla un mensaje de bienvenida */  
  
int main() {  
    cout << "Bienvenido al curso de AEDD empleando C++! ";  
  
    return 0;    /* indica que el programa terminó exitosamente */  
} /* Fin de la funcion main */
```



The screenshot shows a Windows command prompt window with the title bar "C:\Program Files (x86)\Zinja\runner.exe". The window has a black background with white text. The output of the program is displayed as follows:

```
Bienvenido al curso de AEDD empleando C++!  
  
<< El programa ha finalizado: codigo de salida: 0 >>  
<< Presione enter para cerrar esta ventana >>
```

Función main y funciones del usuario

- **Función main**: es el punto de entrada del programa. Hay 1 sola por programa.
- **Función en C++**: subprograma que devuelve un único valor, un conjunto de valores o realiza alguna tarea específica
- La función main de programas extensos generalmente consta de llamadas a otras funciones:

```
int main( )  
{  
    obtenerDatos ( );  
    calcular ( );  
    mostrarDatos ( );  
    return 0;  
}
```

- Las **variables y constantes globales** se declaran fuera de las funciones, generalmente antes de main
- Las **variables y constantes locales** se declaran en la cabecera o cuerpo de cada bloque de función
- Las **funciones de usuario** requieren **declaración o prototipo** (valor de retorno, nombre y lista de parámetros). Se **invocan** por su **nombre** y los **parámetros** que pueda tener. Con la **llamada** se pasa a ejecutar el código de la **función llamada** y al finalizar retorna el control a la **función llamadora**.

Función main y funciones del usuario

```
/* Ejemplo de llamadas a funciones*/  
#include <iostream>  
  
void obtenerDatos ( );  
void calcular ( );  
void mostrarDatos ( );  
  
int main( )  
{  
    obtenerDatos ( );  
    calcular ( );  
    mostrarDatos ( );  
    return 0;  
}  
  
void obtenerDatos ( )  
{  
    cout << " Ingrese los datos " << endl;  
    .....  
}  
.....
```


Creación y Ejecución de un Programa

Etapas de la creación de un programa:

- Definir el programa
- Definir directivas del preprocesador
- Definir declaraciones globales
- Crear main ()
- Crear el cuerpo del programa
- Crear las funciones propias definidas por el usuario
- Utilizar comentarios adecuados
- Almacenar en disco con nombre.cpp
- Compilar, enlazar, ejecutar y comprobar el programa

Estas etapas pueden realizarse en un IDE (entorno de desarrollo integrado)

Ejemplo de un Programa en C++

Se usa el espacio de nombres estandar

Bibliotecas

Constantes simbólicas y literales

Identificadores

Delimitadores

Operadores

Palabras claves

Comentarios

Variables

Entrada/salida

Asignación

```
//- fichero: euros.cpp-  
#include <iostream>  
using namespace std;  
const double EUR_PTS = 166.386;  
int main() {  
  
    cout << "Introduce la cantidad (En euros): ";  
    double euros;  
    cin >> euros;  
    double pesetas = euros * EUR_PTS;  
    cout << euros << " Euros equivalen a " << pesetas << " Pts" << endl;  
    return 0;  
} //- fin: euros.cpp
```

Directivas del Preprocesador

- **Preprocesador**: editor de texto inteligente que procesa **directivas** que son instrucciones al compilador antes que se compile el programa principal.
 - ✓ No son sentencias de C++, por ello no terminan en “;”.
 - ✓ Generalmente se escriben al principio.

- **Las 2 directivas más usuales son:**

#include: indica al compilador que lea del disco el archivo fuente que sigue, y su contenido lo inserte en esa posición. Por ej: `#include <iostream>`.

Los archivos pueden ser de código fuente, diseñado por el usuario, con el formato `<nombre.h>` o ser archivos del sistema, especiales, denominados **archivos de cabecera**, que residen en el compilador: por ejemplo, **<iostream>**, para operaciones de E/S; **<string>**, para operaciones con cadenas.

El nombre del archivo puede darse con formato `<nombre.h>` (si se encuentra en el directorio por defecto) o con formato “path nombre.h” si está en otro directorio. Por ejemplo: `#include “D:\Mis programas\Prueba.h”`.

#define: indica al compilador que defina un ítem de datos. Por ej: `#define PI 3.141` sustituirá PI por el valor 3.141 cada vez que aparezca en el programa.

Elementos básicos de un Programa

Bibliotecas: El lenguaje C++ consta de un reducido número de instrucciones, pero ofrece un amplio repertorio de bibliotecas con herramientas que pueden ser importadas por los programas cuando son necesarias. Por este motivo, un programa suele comenzar por tantas líneas `#include` como bibliotecas se necesiten.

Constantes: Podemos emplear valores en su `sentido literal` (tal y como está escrito), pero también podemos usar constantes simbólicas, definiendo un nombre para referirnos a ellas. Esto último resulta más claro y permitirá que el `programa sea más legible y fácil de modificar en el futuro`.

Antes de utilizar una constante simbólica es necesario informar al compilador de qué `tipo` es la constante, su nombre simbólico (`identificador`) y el `valor` que tiene asociado después del signo (=) y anteponer la palabra reservada `const`.

Por ejemplo: `const int PI=3.141;`

Elementos básicos de un Programa

Identificadores:

Para cada elemento que introduzcamos en nuestro programa debemos definir un **nombre (identificador)** con el que hacer referencia al mismo y disponer de algún mecanismo para informar al compilador de dicho nombre y de sus características.

En C++ se considera que las letras minúsculas son caracteres diferentes de las letras mayúsculas.

Un **identificador** debe estar formado por una secuencia de letras y dígitos en la que el primer carácter debe ser una letra.

- El carácter '_' puede utilizarse, sin embargo, los nombres no deben comenzar con dicho carácter.
- Son sensibles a las mayúsculas.
- Pueden ser extensos, pero son significativos los 32 primeros caracteres.
- No pueden usarse palabras reservadas.

Ejemplos de identificadores:

nombre_cliente

dni

Mayor_item

fechaInicio

Elementos básicos de un Programa

Palabras reservadas:

Algunas palabras tienen un significado especial en el lenguaje y no pueden ser utilizadas con otro sentido.

Por eso no pueden ser utilizadas para designar elementos definidos por el programador.

Son palabras reservadas, por ejemplo: `using`, `namespace`, `const`, `float`, `double`, `int`, `char`, `bool`, `void`, `for`, `while`, `do`, `if`, `switch`, `case`, `default`, `return`, `typedef`, `struct`, entre otras.

Delimitadores:

Son símbolos que indican comienzo o fin de una entidad: `( ) { } ; , < >`.

Por ejemplo, en nuestro programa `euros.cpp` usamos `{` y `}` para delimitar el comienzo y el final de la función *main*, y el símbolo `;` para delimitar el final de una sentencia.

Operadores:

Son símbolos con significado propio según el contexto en el que se utilicen. Ejemplo: `= << >> * / % + - < > <= >= == != ++ -- . ,`, etc.

Elementos básicos de un Programa

Comentarios y formato del programa:

El compilador necesita reconocer los diferentes elementos que forman el programa. Para ello, utiliza **delimitadores y caracteres** que permiten separar unos elementos de otros.

Además, el programador puede añadir caracteres como espacios en blanco, líneas en blanco o tabuladores para **mejorar la legibilidad del programa**.

Además, el programador puede estar interesado en mejorar la legibilidad del programa incluyendo **comentarios dirigidos a otros programadores**, no al compilador.

Para ello, debe marcar dicho texto de alguna forma que permita ser identificado por el compilador. En C++ esto se puede hacer de dos formas diferentes:

- **Comentarios hasta fin de línea:** los símbolos `//` marcan el comienzo del comentario, que se extiende hasta el final de la línea.
- **Comentarios enmarcados:** los símbolos `/*` marcan el comienzo del comentario, que se extiende hasta los símbolos de fin del comentario `*/`.

Elementos básicos de un Programa

Variables:

Los datos se almacenan en memoria en variables de un cierto tipo. El programador debe decidir qué variables va a utilizar, pensar en un identificador para referirse a ellas y comunicarlo al compilador.

Entrada/Salida de datos:

En general, un programa necesitará tomar datos de entrada y mostrar resultados en algún dispositivo de salida.

En C++ disponemos de flujos de entrada (**cin**) y de salida (**cout**), que nos permiten efectuar entrada y salida de datos, respectivamente. El operador **>>** se usa para tomar un valor de la entrada y el operador **<<** se usa para sacar un valor por la salida.

Asignación:

Si queremos realizar un cálculo y almacenar el resultado de manera que pueda ser utilizado posteriormente, utilizamos una instrucción de asignación en la que el operador **=** separa el valor a almacenar, resultado de evaluar una expresión aritmética (a la derecha del signo **=**) de la variable en la que lo queremos almacenar (asignar).

Depuración de un Programa

Se denomina así al proceso de encontrar errores. Los errores pueden ser:

- **Errores de Sintaxis:** Se violan las reglas de gramática del lenguaje. Ej: punto y coma después de la cabecera; omisión de “;” al final de una sentencia; olvido de símbolos apareados (corchetes, paréntesis, /*..).
- **Errores Lógicos:** Errores del programador en el diseño del algoritmo y posterior programa. No los detecta el compilador. Se detectan con cuidadoso análisis de los resultados (testing). Se puede necesitar un depurador (debugger).
- **Errores de Ejecución** se presentan después que el programa fue compilado, y cuando está en ejecución. Ej: Error por desbordamiento de una variable; error de división por cero.

Los compiladores emiten **mensajes de error o de advertencia:**

- ✓ Los mensajes de error del compilador suelen deberse a **errores de sintaxis** y son **fatales**: no puede realizar la compilación, el compilador avisa el error y termina.
- ✓ **Advertencia** (warning): indican condiciones legítimas pero sospechosas.

Pruebas

- **Los Errores de Ejecución** se presentan después que el programa fue compilado, y cuando está en ejecución.
- El compilador puede no emitir ningún mensaje de error durante la ejecución, y eso no garantiza que el programa sea correcto pues solo indica que un programa se escribió bien sintácticamente. No indica si el programa hace lo que se desea que haga (**eficaz**).
- Para determinar si un programa tiene un error lógico, se debe ejecutar utilizando **datos de muestra** y comprobar la salida (**prueba – testing**). Esta prueba se ejecuta con diferentes entradas preparadas y seleccionadas preferentemente por personas diferentes al programador.
- Si se ha determinado que el programa tiene un error lógico y no se lo encuentra fácilmente, para localizarlo se debe realizar la **ejecución paso a paso**, **seguir la traza**. Algunos compiladores tienen un depurador (debugger) incorporado con el editor.

Máxima de Dijkstra (científico holandés): *Las pruebas sólo muestran **la presencia** de errores, **no su ausencia**. No se puede probar que un programa es correcto (exacto), sólo se puede mostrar que es incorrecto.*

LEER

❑ Capítulo 1

Libro: “Fundamentos de Programación con el Lenguaje de Programación C++” – Benjumea-Roldan.

❑ Capítulos 1 y 2

Libro: “C++ para ingeniería y ciencias” – Gary J. Bronson, 2da Edición.

❑ Capítulos 1 y 2

Libro: “Programación y resolución de problemas con C++” – Nell Dale - Chip Weems.